



Java While Loop

[< Previous](#)[Next >](#)

Loops

Loops can execute a block of code as long as a specified condition is true.

Loops are handy because they save time, reduce errors, and they make code more readable.

Java While Loop

The **while** loop repeats a block of code as long as the specified condition is **true**:

Syntax

[Get your own Java Server](#)

```
while (condition) {  
    // code block to be executed  
}
```

In the example below, the code in the loop will run again and again, as long as a variable (**i**) is less than 5:

Example

```
int i = 0;  
while (i < 5) {  
    System.out.println(i);  
}
```

[Try it Yourself »](#)

Note: Do not forget to increase the variable used in the condition (`i++`), otherwise the loop will never end!

Do you wonder why we used the letter `i` in the example above? It's a **counter** variable and a common choice in simple loops because it's short, traditional, and stands for 'index' or 'iterator'.

Countdown Example

You can also use a `while` loop to count down. This example counts from 3 to 1, and then prints "Happy New Year!!" at the end:

Example

```
int countdown = 3;

while (countdown > 0) {
    System.out.println(countdown);
    countdown--;
}

System.out.println("Happy New Year!!");
```

[Try it Yourself »](#)[REMOVE ADS](#)



times. But if the condition is **false** at the beginning, the code inside the loop will never run:

Example

```
int i = 10;

while (i < 5) {
    System.out.println("This will never be printed");
    i++;
}
```

Try it Yourself »

Note: A **while** loop may never run if the condition is false from the start. In the next chapter, you will learn about the **do while** loop, which always runs the code at least once before checking the condition.

Exercise [?]

Drag and drop the correct keyword to start the loop.

```
int i = 0;
    while    (i < 3) {
    System.out.println(i);
    i++;
}
```

for if switch